GH-300 · Exam-Day Cheat Sheet

Read once, slowly. Then walk in. No new material. ✨

🗓️ Fri Aug 28, 2026 · 🎯 Pass 700/1000 · ⏱️ ~8 min read · 📊 Scope: Aug 7, 2026

> **If two options both look right** — pick the one that matches the exact wording of the stem. The exam rewards discriminator words, not depth.

① The five rules that decide most questions

1. **Human accountability never transfers.** Copilot output is a draft: review, test, validate. A filter reduces risk — it never certifies correctness, security, or licensing.
2. **Who is the customer?** Individual → Free / Pro / Pro+. Organization → Business / Enterprise.
3. **Governance starts at Business:** org policies, content exclusions, audit logs, seat management, no-training commitment, conditional IP indemnity. **Enterprise** = Business + GitHub Enterprise Cloud + knowledge bases.
4. **Where does the work run?** Editor + synchronous → completions / ask / Edits / agent mode. GitHub + asynchronous → coding agent.
5. **Copilot advises, humans gate.** Copilot code review never approves; branch protection and rulesets enforce human review.

② Which mode? Discriminator words

STEM PHRASE	ANSWER
"as I type", ghost text, no prompt	**Code completions**
"explain it, I'll apply it myself"	**Ask mode**
"I know the files, change them together"	**Copilot Edits**
multi-step, picks files, runs commands, *locally*, "I watch"	**Agent mode**
"assign an issue", @copilot, background, draft PR	**Coding agent**

Lock: "locally" + "I'll watch" = agent mode. "Assigned issue" + async + draft PR = coding agent — and it cannot approve its own PR.

③ Which context surface?

NEED	ANSWER
Always-on project rules for a whole repo	**Repo custom instructions**
Reusable, on-demand task	**Prompt file**
Live external data or tools	**MCP server / extension**
Reusable bundle of repos, files, notes	**Copilot Space**
Curated internal docs on github.com	**Knowledge base (Enterprise)**
One-off targeted context	`#file` `#selection`

Lock: `@` routes expertise · `#` attaches context · `/` shortcuts a task.

④ Suggestion lifecycle — draw it

```
IDE gathers context
  (current file, cursor, neighboring tabs,
   custom instructions, exclusion policy)
  ↓ prompt assembled client-side, HTTPS
      (the model does NOT run locally)
  ↓ PROXY: exclusions + INPUT filter
      (prompt injection, toxic language)
  ↓ hosted model generates
  ↓ OUTPUT filters: public-code match (~150 ch),
      harmful content, insecure patterns,
      empty / repetitive discard
  ↓ suggestion shown → accept / reject → telemetry
```

Retention: Business/Enterprise code is **never** used for training · completions are discarded after return · **Chat is retained** for conversation context · individual plans may be used unless opted out. "Not trained on" ≠ "no telemetry".

⑤ Prompt engineering

Anatomy: goal + context + expectations + source of truth.

- Zero-shot = no example · One-shot = one · **Few-shot = several** (best for internal patterns absent from public code).
- Bad answer? Diagnose in order: **under-specified → over-scoped → missing context**.
- Generic output → few-shot examples or custom instructions.
- Wrong file → attach it explicitly.
- Drifted after many turns → new thread, restate constraints.

⑥ Productivity plays

```
Legacy code, no tests → explain → characterization tests FIRST
                          → refactor incrementally
Need tests            → happy path → edge cases, boundaries, errors
Need docs             → generate from code → verify real behavior
Security finding      → fix + explanation → validate
                        (CodeQL autofix is separate)
Performance issue     → measure → targeted fix → re-measure
Deprecated API        → re-prompt with target version → check docs
```

⑦ Privacy & safeguards

Content exclusion does 4 things: no inline suggestions in the file · the file does not inform suggestions elsewhere · it does not inform Chat · it is not reviewed by Copilot code review.

The 5 leak paths 🔓

1. **Edit mode and Agent mode** of Chat in VS Code and other editors
2. **Semantic info from the IDE** — types, hover definitions, build config
3. **Symlinks**
4. **Remote filesystems**
5. **Not retroactive, not a security boundary** — identical content elsewhere can still surface

Who configures: repo admins, org owners, enterprise owners. **Plans:** Business and Enterprise only. github.com and Mobile = public preview. Xcode/Eclipse = inline only, no Chat. Azure Data Studio = unsupported. The client sends the repo URL to fetch the policy; those URLs are not logged.

IP indemnity is CONDITIONAL on the duplication-detection / public-code filter being enabled — disable it and coverage is forfeited.
Public ≠ free: licenses may impose copyleft; no license = all rights reserved.
Hierarchy: enterprise policies cannot be overridden by org owners.

⑧ Numbers & weights

Public-code match window	**~150 chars**
Activity report retention	**90 days**
Copilot audit log retention	**180 days**
Copilot Memory unused facts	**28 days**
Exclusion propagation to loaded IDEs	**30 min**

DOMAIN	WEIGHT
D2 Copilot features	**25–30%**
D1 Responsible use	**15–20%**
D3 Data & architecture	**10–15%**
D4 Prompt engineering	**10–15%**
D5 Productivity	**10–15%**
D6 Privacy & safeguards	**10–15%**

⑨ Your personal trap list 🎯

- Content exclusion limits — recall **the 5 leak paths**, not generic controls.
- **Business** is the lowest tier with org policies, exclusions, and audit logs. Pro+ is still individual.
- Agent mode vs coding agent — decide by *locally / synchronous* vs *assigned issue / async*.
- Indemnity requires the public-code filter to be **ON**.

Day-2 results: warm-up 3/4 · exclusion drill 5/5 · D3 6/6 · D4 4/4 · D5 3/3 · **mock #2 10/10**. You're ready. 💪

Sources: Microsoft Learn GH-300 study guide · GitHub Docs (plans, code suggestions, code referencing, prompt engineering, content exclusion, policies, audit logs, coding agent, code review).
Made for one nervous morning. ♥